CS543 Web Services - Assignment 2

Design: Services, Contracts and Schema - Ecommerce (bags store) MCA,
IIIT Vadodara | Date: 13-10-26

Ecommerce is a MERN web store (MongoDB, Express, React, Node) where customers buy backpacks, laptop bags and sling bags. Payments go through Razorpay in test mode. Roles are customer and admin. This document is the design benchmark: capabilities, services, contracts, the placeOrder specification and the validation of the services.

1. Capabilities (verbs from the Assignment 1 brief)

#	Capability	Who	Service
1	Register an account	Customer	Auth
2	Log in / log out	Customer, Admin	Auth
3	Browse, search and filter bags, view one bag	Customer	Catalogue
4	Manage cart (add, remove, change quantity)	Customer	Order
5	Place order	Customer	Order
6	View my orders	Customer	Order
7	Cancel order	Customer	Order
8	Create Razorpay order (start payment)	System	Payment
9	Verify payment (signature check)	System	Payment
10	Add / edit / delete a bag	Admin	Catalogue
11	Update order status (packed, shipped, delivered)	Admin	Order

2. Services, members and data owned

Capabilities are grouped by the data they work on. Each service owns its own data and no two services share data. Other services can only reach that data through the contract operations in section 3.

Service	Member	Owns data	Calls
Auth	Abhishek,Binamra,Navendu	User accounts, login data, roles	None
Catalogue	Shruti,Yatindra	Products, categories, stock count	Auth: verifyToken (admin only operations)
Order	Navendu,Yatindra,Shruti	Cart, addresses, orders, order items	Auth: verifyToken Catalogue: checkStock, reduceStock, restoreStock Payment: createPayment, verifyPayment
Payment	Binamra rana, Abhishek,Yatindra	Payment records (Razorpay order id, payment id, signature, status)	Razorpay (external API)

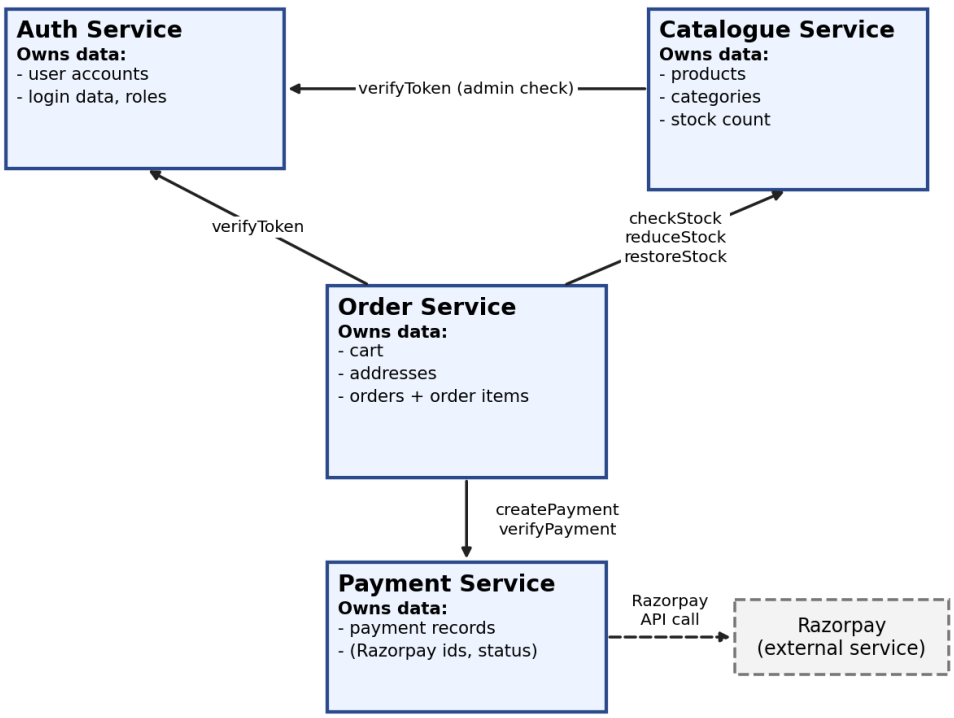


Figure: service design diagram (editable source: services.drawio).

3. Contracts

Each table lists the operations other services may call. Inputs and outputs are plain data; the inside of the service is hidden.

3.1 Auth Service (Member: Abhishek,Binamra rana,Navendu)

Operation	Input	Output	Errors
register	name, email, password	userId, message	EMAIL_ALREADY_USED (409), INVALID_INPUT (400)
login	email, password	token, userId, role	WRONG_CREDENTIALS (401)
verifyToken	token	userId, role	TOKEN_INVALID (401), TOKEN_EXPIRED (401)

3.2 Catalogue Service (Member: Shruti,Yatindra)

Operation	Input	Output	Errors
listProducts	category (optional), search (optional)	list of {productId, name, price, category, image, stock}	None (empty list if nothing matches)
getProduct	productId	productId, name, description, price, category, image, stock	PRODUCT_NOT_FOUND (404), INVALID_ID (400)
checkStock	productId, quantity	available (true/false), name, price	PRODUCT_NOT_FOUND (404)
reduceStock	productId, quantity	newStock	OUT_OF_STOCK (409), PRODUCT_NOT_FOUND (404)
restoreStock	productId, quantity	newStock	PRODUCT_NOT_FOUND (404)
addProduct	token, name, price, category, image, stock	productId	NOT_ADMIN (403), INVALID_INPUT (400)
updateProduct	token, productId, fields to change	updated product	NOT_ADMIN (403), PRODUCT_NOT_FOUND (404)
deleteProduct	token, productId	message	NOT_ADMIN (403), PRODUCT_NOT_FOUND (404)

3.3 Order Service (Member: Navendu,Yatindra,Shruti)

Operation	Input	Output	Errors
placeOrder	token, addressId	orderId, status, totalAmount, razorpayOrderId, razorpayKeyId	See section 4
confirmPayment	token, orderId, razorpayOrderId, razorpayPaymentId, signature	orderId, status = PAID	ORDER_NOT_FOUND (404), PAYMENT_VERIFICATION_FAILED (400)
getMyOrders	token	list of {orderId, items, totalAmount, status, date}	TOKEN_INVALID (401)

cancelOrder	token, orderId	orderId, status = CANCELLED	ORDER_NOT_FOUND (404), NOT_YOUR_ORDER (403), CANNOT_CANCEL (409) if already shipped

Operation	Input	Output	Errors
updateOrderStatus	token, orderId, newStatus	orderId, status	NOT_ADMIN (403), ORDER_NOT_FOUND (404), INVALID_STATUS (400)
addToCart	token, productId, quantity	cart	PRODUCT_NOT_FOUND (404), INVALID_QUANTITY (400)
changeCartItem	token, productId, quantity	cart	ITEM_NOT_IN_CART (404), INVALID_QUANTITY (400)
removeFromCart	token, productId	cart	ITEM_NOT_IN_CART (404)
getCart	token	list of {productId, name, price, quantity}, total	TOKEN_INVALID (401)

3.4 Payment Service (Member: Binamra rana, Abhishek,Yatindra)

Operation	Input	Output	Errors
createPayment	orderId, amount (rupees)	razorpayOrderId, amount in paise, razorpayKeyId	INVALID_AMOUNT (400), GATEWAY_UNAVAILABLE (502)
verifyPayment	orderId, razorpayOrderId, razorpayPaymentId, signature	verified (true/false)	PAYMENT_NOT_FOUND (404), SIGNATURE_MISMATCH (400)

4. Central operation: placeOrder

Owner: Order Service. **Purpose:** turn the customer's cart into an order that is waiting for payment. The caller only sees the inputs, outputs and errors below.

Inputs

Name	Meaning
token	Login token of the customer
addressId	Id of the saved delivery address chosen by the customer. Items come from the customer's own cart, the caller does not send prices.

Outputs (on success)

Name	Meaning
orderId	Id of the new order
status	PENDING_PAYMENT
totalAmount	Total in rupees, calculated by the server
razorpayOrderId, razorpayKeyId	Needed by the frontend to open the Razorpay payment window

Error cases

Error	When	What the caller gets
TOKEN_INVALID (401)	Token missing, wrong or expired	No order created
CART_EMPTY (400)	Customer has nothing in the cart	No order created
ADDRESS_NOT_FOUND (404)	addressId does not exist or belongs to someone else	No order created
PRODUCT_NOT_FOUND (404)	A bag in the cart was removed from the shop	No order created, message names the bag
OUT_OF_STOCK (409)	Stock is less than the quantity asked	No order created, message names the bag
PAYMENT_UNAVAILABLE (502)	Razorpay could not create a payment	No order left behind, stock given back

Internal details hidden from callers

1. Token is checked by calling Auth (verifyToken). The caller never talks to Auth for this.
2. The cart is read and, for every bag, Catalogue is asked for the current price and stock (checkStock).

Prices sent by the browser are never trusted.

3. Total is calculated on the server (price x quantity, added up).
4. Stock is reduced (reduceStock) for every bag. If one bag fails, the bags already reduced are given back(restoreStock).
5. The order and its items are saved with a copy of the name and price at that time, so old orders do notchange if the price changes later.
6. Payment is asked to create a Razorpay order (createPayment). Rupees are converted to paise there. Ifthis fails, the order is cancelled and stock is restored.
7. The cart is cleared only after the payment is confirmed (confirmPayment), not here.
8. Result is all-or-nothing: the caller either gets a complete pending order or an error with nothing lefthalf-done.

5. Database schema

The ER diagram is in schema.drawio / schema.png and the CREATE TABLE sketch is in schema.sql.

Eight tables: users (Auth); categories, products (Catalogue); cart_items, addresses, orders, order_items (Order); payments (Payment). No table belongs to two services. Ids that point to another service (user_id, product_id, order_id) are plain columns, not foreign keys.

6. Validation against the five properties

Pass = fine, Partly = works but has a weakness, Fail = not met yet.

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled
Auth	Pass	Pass	Pass	Pass	Pass
Catalogue	Partly	Pass	Fail	Pass	Pass
Order	Partly	Pass	Pass	Fail	Partly
Payment	Partly	Pass	Pass	Pass	Pass

Where it fails, and the fix

Property	Problem	Fix
Reachable (Catalogue, Order, Payment)	At the moment all services run inside one Express app on port 5000. They are reachable by URL (/api/products, /api/orders, /api/payment) but not as separate programs.	Keep each service in its own routes/controller folder and talk only through its URL. If time allows, run each on its own port.
Has a contract (Catalogue)	In the current code POST /api/products has no admin check (comment says it will be added later). So the NOT_ADMIN error in the contract is not real yet.	Add a check that calls Auth verifyToken and allows only role = admin before add/update/delete.
Independent (Order)	Order cannot place an order when Catalogue or Payment is down, because it needs stock and a Razorpay order.	Return a clear error (PAYMENT_UNAVAILABLE / PRODUCT_NOT_FOUND), restore stock, and let the user retry. Cart and old orders still work without the other services.
Loosely coupled (Order)	Order is the busiest caller (Auth, Catalogue, Payment) and calls them one after the other. A change in the checkStock output could break placeOrder.	Order uses only the contract operations and never reads another service's data. Keep operation names and outputs stable. Later, Payment could tell Order by an event instead of Order asking.